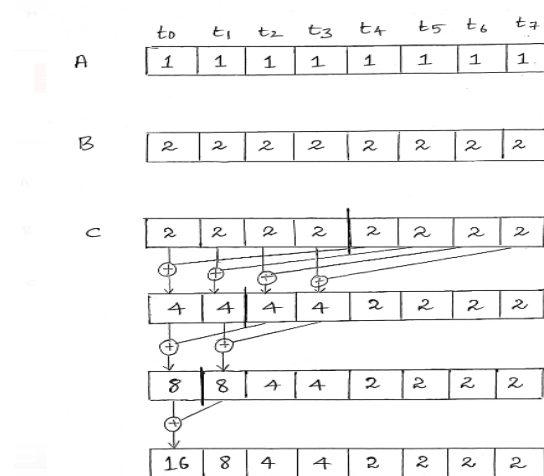


1. Dot product of two vectors using parallel reduction



Dot product = 16

```
#include<stdio.h>
#include<cuda.h>
#define N 16
__global__ void multiply(int *a,int *b,int *c)
{
    int i=threadIdx.x;
    if (i < N)
        c[i]=a[i] * b[i];
}

__global__ void add(int *d,int *e)
{
    int t = threadIdx.x;
    int psize=N/2;
    while(psize != 0 )
    {
        d[t] =d[t] + d[t+psize];
        psize = psize /2;
        __syncthreads();
    }
    if (t==0)
        *e = d[0];
}

int main()
{
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    int *a,*b,*c,*d,i;
```

```

int size=N * sizeof(int);

a = (int*)malloc(size);
b = (int*)malloc(size);
c = (int*)malloc(size);
d = (int*)malloc(size);

int *da,*db,*dc,*dd;
cudaMalloc((void**)&da,size);
cudaMalloc((void**)&db,size);
cudaMalloc((void**)&dc,size);
cudaMalloc((void**)&dd,size);

for (i=0;i<N;i++)
{
    a[i] = 1;
    b[i] = 2;
}

cudaMemcpy(da,a,size,cudaMemcpyHostToDevice);
cudaMemcpy(db,b,size,cudaMemcpyHostToDevice);

cudaEventRecord(start, 0);

multiply<<<1,N>>>(da,db,dc);
add<<<1,N/2>>>(dc,dd);

    cudaEventRecord(stop,0);
cudaEventSynchronize(stop);

cudaMemcpy(d,dd,size,cudaMemcpyDeviceToHost);

printf("\n Dot Product of two vectors: %d\n",d[0]);

float elapsedtime;
cudaEventElapsedTime(&elapsedtime, start, stop);
printf("CUDA Execution Time = %f ms", elapsedtime);

free(a);
free(b);
free(c);
free(d);

cudaFree(da);
cudaFree(db);
cudaFree(dc);
cudaFree(dd);

```

```

    cudaEventDestroy(start);
    cudaEventDestroy(stop);

    return 0;
}

```

Output

```

[vk@Prithivi ~]$ nvcc ex1_time.cu
[vk@Prithivi ~]$ ./a.out

```

Dot Product of two vectors: 32
 CUDA Execution Time = 0.051456 ms

2. Matrix Transpose using shared memory

```

#include<stdio.h>
#define ROW 4
#define COL 4
__global__ void mtrans_smem(int *a, int *b)
{

    __shared__ int tile[ROW/2][COL/2];

    int x = threadIdx.x + blockIdx.x * blockDim.x;
    int y = threadIdx.y + blockIdx.y * blockDim.y;
    int from = y * COL + x;

    int bidx = threadIdx.y * blockDim.x + threadIdx.x;
    int irow = bidx / blockDim.y;
    int icol = bidx % blockDim.y;

    // coordinate in transposed matrix
    x = blockIdx.y * blockDim.y + icol;
    y = blockIdx.x * blockDim.x + irow;

    // linear global memory index for transposed matrix
    int to = y * ROW + x;

    // transpose with boundary test
    if (x < COL && y < ROW)
    {
        // load data from global memory to shared memory
        tile[threadIdx.y][threadIdx.x] = a[from];

        // thread synchronization
    }
}

```

```

        __syncthreads();

        // store data to global memory from shared memory
        b[to] = tile[icol][irow];
    }
}

int main()
{

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    int *a, *b;
    int i, j;
    int size = ROW * COL * sizeof(int);

    a = (int*)malloc(size);
    b = (int*)malloc(size);

    int *d_a, *d_b;
    cudaMalloc((void**)&d_a, size);
    cudaMalloc((void**)&d_b, size);

    for(i=0; i<ROW*COL; i++)
    {
        a[i] = i;
    }

    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);

    dim3 block(2,2);
    dim3 grid(2,2);

    cudaEventRecord(start, 0);

    mtrans_smem<<<grid, block>>>(d_a, d_b);

    cudaEventRecord(stop,0);
    cudaEventSynchronize(stop);

    cudaMemcpy(b, d_b, size, cudaMemcpyDeviceToHost);

    for(i=0; i<ROW; i++)
    {
        for(j=0; j<COL; j++)
        {
            printf("\t %d", b[i * COL + j]);

```

```

    }
    printf("\n");
}

```

```

float elapsedtime;
cudaEventElapsedTime(&elapsedtime, start, stop);
printf("CUDA Execution Time = %f ms", elapsedtime);

```

```

cudaFree(d_a);
cudaFree(d_b);

```

```

free(a);

```

```

free(b);

```

```

cudaEventDestroy(start);
cudaEventDestroy(stop);

```

```

return 0;
}

```

Output

```

[vk@Prithivi ~]$ nvcc ex2_time.cu

```

```

[vk@Prithivi ~]$ ./a.out

```

```

    0    4    8   12
    1    5    9   13
    2    6   10   14
    3    7   11   15

```

```

CUDA Execution Time = 0.041792 ms

```

3. 1D stencil using constant memory

```

#include<stdio.h>

```

```

#define N 16

```

```

#define RADIUS 2

```

```

#define BLOCK_SIZE 8

```

```

__constant__ int d_coef[RADIUS+1];

```

```

__global__ void stencil_1d_smem(int *in, int *out)

```

```

{

```

```

    __shared__ int sm[BLOCK_SIZE + 2 * RADIUS];

```

```

    int gindex = threadIdx.x + blockIdx.x * blockDim.x;

```

```

    int lindex = threadIdx.x + RADIUS;
    sm[lindex] = in[gindex]; //storing in shared memory

    if (threadIdx.x < RADIUS)
    {
        int tmp = gindex - RADIUS;
        if(tmp < 0)
            sm[lindex-RADIUS] = 0;
        else
            sm[lindex - RADIUS] = in[gindex - RADIUS];
        sm[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];
    }
    __syncthreads();

    int result = 0;
    int i;
    for(i=1;i<=RADIUS;i++)
    {
        result += d_coef[i] * (sm[lindex+i] - sm[lindex-i]);
    }
    out[gindex] = result;
}

int main()
{
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    const int coef[] = {0,1,2};
    cudaMemcpyToSymbol(d_coef, coef, (RADIUS + 1) * sizeof(int));
    int i;
    int in[N + 2 * RADIUS], out[N];

    int *d_in, *d_out;
    cudaMalloc((void**)&d_in, (N + 2 *RADIUS)*sizeof(int));
    cudaMalloc((void**)&d_out, N * sizeof(int));

    for( i = 0; i < (N + 2 * RADIUS); i++ )
        in[i] = i;

    cudaMemcpy(d_in, in, (N + 2 * RADIUS) * sizeof(int), cudaMemcpyHostToDevice);

    cudaEventRecord(start, 0);

    stencil_1d_smem<<< (N + BLOCK_SIZE - 1)/BLOCK_SIZE, BLOCK_SIZE >>> (d_in,
d_out);

```

```

cudaEventRecord(stop,0);
cudaEventSynchronize(stop);

cudaMemcpy(out, d_out, N * sizeof(int), cudaMemcpyDeviceToHost);

printf("\n");
for(i=0; i< N; i++)
    printf("\t %d ",in[i]);

printf("\n");
for(i=0; i<N; i++)
    printf("\t %d ",out[i]);

float elapsedtime;
cudaEventElapsedTime(&elapsedtime, start, stop);
printf("\n CUDA Execution Time = %f ms", elapsedtime);

cudaFree(d_in);
cudaFree(d_out);

cudaEventDestroy(start);
cudaEventDestroy(stop);

return 0;
}

```

Output

```

[vk@Prithivi ~]$ nvcc ex3_time.cu
[vk@Prithivi ~]$ ./a.out

```

```

Input array:  0   1   2   3   4   5   6   7   8   9   10  11  12  13
14  15
Output array:  5   8  10  10  10  10  10  10  10  10  10  10  10  10
10  10  10
CUDA Execution Time = 0.041408 ms

```

4. Odd Even Transposition sort

```

#include<stdio.h>
#define N 6
__global__ void OE_sort(int *a,int n)
{
    int tid=threadIdx.x;
    int i,t;
    for(i=1;i<=n/2;i++)

```

```

{
    if(tid < n-1 && tid % 2 == 1)
    {
        t = a[tid+1];
        a[tid+1] = max(a[tid],t);
        a[tid] = min(a[tid],t);
    }
    __syncthreads();

    if(tid < n-1 && tid % 2 == 0)
    {
        t = a[tid+1];
        a[tid+1] = max(a[tid],t);
        a[tid] = min(a[tid],t);
    }
    __syncthreads();
}
}
int main()
{

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    int a[]={70,80,60,40,50,30};
    int i;

    printf("\n Array elements before sorting:");
    for(i=0;i<N;i++)
        printf("\t %d",a[i]);

    int *d_a;
    cudaMalloc((void**)&d_a,N*sizeof(int));

    cudaMemcpy(d_a,&a,N*sizeof(int),cudaMemcpyHostToDevice);

    cudaEventRecord(start, 0);

    OE_sort<<<1,N>>>(d_a,N);

    cudaEventRecord(stop,0);
    cudaEventSynchronize(stop);

    cudaMemcpy(&a,d_a,N*sizeof(int),cudaMemcpyDeviceToHost);

    printf("\n Array elements after sorting:");
    for(i=0;i<N;i++)

```

```

    printf("\t %d",a[i]);

float elapsedtime;
cudaEventElapsedTime(&elapsedtime, start, stop);
printf("\n CUDA Execution Time = %f ms", elapsedtime);

cudaFree(d_a);

cudaEventDestroy(start);
cudaEventDestroy(stop);

return 0;
}

```

Output

```

[vk@Prithivi ~]$ vi ex4_time.cu
[vk@Prithivi ~]$ nvcc ex4_time.cu
[vk@Prithivi ~]$ ./a.out

```

```

Array elements before sorting: 70    80    60    40    50    30
Array elements after sorting:  30    40    50    60    70    80
CUDA Execution Time = 0.046944 ms

```

5. Pre-order Tree Traversal

```

#include <stdio.h>
/* input tree
    a
   / \
  b  c
 / \ /
d  e f
 / \ /
g  h i
*/
__device__ int temp[9][9];

__global__ void traverse(int *parent,int *child,int *sibling,int *edge0,int *edge1,int
*succ0,int *succ1,int *position,int *preorder) {
int i=threadIdx.x;
if(parent[edge0[i]]==edge1[i]){
    if(sibling[edge0[i]]!=-1){
        succ0[i]=edge1[i];
        succ1[i]=sibling[edge0[i]];
    }
}
}

```

```

    else if(parent[edge1[i]]!=-1){
        succ0[i]=edge1[i];
        succ1[i]=parent[edge1[i]];
    }
    else{
        succ0[i]=edge0[i];
        succ1[i]=edge1[i];
        preorder[edge1[i]]=1;
    }
}
else{
    if(child[edge1[i]]!=-1){
        succ0[i]=edge1[i];
        succ1[i]=child[edge1[i]];
    }
    else{
        succ0[i]=edge1[i];
        succ1[i]=edge0[i];
    }
}
}

if(parent[edge0[i]]==edge1[i]){
    position[i]=0;
}
else{
    position[i]=1;
}
int x;
for(int k=0;k<4;k++){
    x=temp[succ0[i]][succ1[i]];
    position[i]=position[i]+position[x];
    succ0[i]=succ0[x];
    succ1[i]=succ1[x];
}

if(edge0[i]==parent[edge1[i]]){
    preorder[edge1[i]]=9+1-position[i];
}

}

__global__ void initialize(int *edge0,int *edge1){
    for(int i=0;i<16;i++){
        temp[edge0[i]][edge1[i]]=i;
    }
}

int main()
{

```

```

cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

char vertices[9]={'a','b','c','d','e','f','g','h','i'};
int parent[9]={-1,0,0,1,1,2,3,3,4};
int child[9]={1,3,5,6,8,-1,-1,-1,-1};
int sibling[9]={-1,2,-1,4,-1,-1,7,-1,-1};
int edge0[16]={0,1,1,3,3,6,3,7,1,4,0,2,4,8,2,5};
int edge1[16]={1,0,3,1,6,3,7,3,4,1,2,0,8,4,5,2};
int succ0[16]; int succ1[16]; int position[16]; int preorder[9];
int *dparent,*dchild,*dsibling,*dedge0,*dedge1,*dsucc0,*dsucc1,*dposition,*dpreorder;

cudaMalloc((void**)&dparent,9*sizeof(int));
cudaMalloc((void**)&dchild,9*sizeof(int));
cudaMalloc((void**)&dsibling,9*sizeof(int));
cudaMalloc((void**)&dedge0,16*sizeof(int));
cudaMalloc((void**)&dedge1,16*sizeof(int));
cudaMalloc((void**)&dsucc0,16*sizeof(int));
cudaMalloc((void**)&dsucc1,16*sizeof(int));
cudaMalloc((void**)&dposition,16*sizeof(int));
cudaMalloc((void**)&dpreorder,9*sizeof(int));

cudaMemcpy(dparent,&parent,9*sizeof(int),cudaMemcpyHostToDevice);
cudaMemcpy(dchild,&child,9*sizeof(int),cudaMemcpyHostToDevice);
cudaMemcpy(dsibling,&sibling,9*sizeof(int),cudaMemcpyHostToDevice);
cudaMemcpy(dedge0,&edge0,16*sizeof(int),cudaMemcpyHostToDevice);
cudaMemcpy(dedge1,&edge1,16*sizeof(int),cudaMemcpyHostToDevice);
cudaMemcpy(dsucc0,&succ0,16*sizeof(int),cudaMemcpyHostToDevice);
cudaMemcpy(dsucc1,&succ1,16*sizeof(int),cudaMemcpyHostToDevice);
cudaMemcpy(dposition,&position,16*sizeof(int),cudaMemcpyHostToDevice);
cudaMemcpy(dpreorder,&preorder,9*sizeof(int),cudaMemcpyHostToDevice);

cudaEventRecord(start, 0);

initialize<<<1,1>>>(dedge0,dedge1);
traverse<<<1,16>>>(dparent,dchild,dsibling,dedge0,dedge1,dsucc0,dsucc1,dposition
,dpreorder);

cudaEventRecord(stop,0);
cudaEventSynchronize(stop);

cudaMemcpy(&succ0,dsucc0,16*sizeof(int),cudaMemcpyDeviceToHost);
cudaMemcpy(&succ1,dsucc1,16*sizeof(int),cudaMemcpyDeviceToHost);
cudaMemcpy(&preorder,dpreorder,9*sizeof(int),cudaMemcpyDeviceToHost);

printf("Preorder Traversal numbering to the vertices: \n");
for(int i=0;i<9;i++){

```

```

    printf("%c -> %d\n",vertices[i],preorder[i]);
}

float elapsedtime;
cudaEventElapsedTime(&elapsedtime, start, stop);
printf("\n CUDA Execution Time = %f ms", elapsedtime);

cudaFree(dparent);
cudaFree(dchild);
cudaFree(dsibling);
cudaFree(dedge0);
cudaFree(dedge1);
cudaFree(dsucc0);
cudaFree(dsucc1);
cudaFree(dposition);
cudaFree(dpreorder);

cudaEventDestroy(start);
cudaEventDestroy(stop);

return 0;
}

```

Output

```

[vk@Prithivi ~]$ vi ex5_time.cu
[vk@Prithivi ~]$ nvcc ex5_time.cu
[vk@Prithivi ~]$ ./a.out
Preorder Traversal numbering to the vertices:
a -> 1
b -> 2
c -> 8
d -> 3
e -> 6
f -> 9
g -> 4
h -> 5
i -> 7

```

CUDA Execution Time = 0.087264 ms

6. Quick sort

```

#include <stdio.h>

__device__ int d_size;

```

```

__global__ void partition (int *arr, int *arr_l, int *arr_h, int n)
{
    int z = blockIdx.x*blockDim.x+threadIdx.x;
    d_size = 0;
    __syncthreads();
    if (z<n)
    {
        int h = arr_h[z];
        int l = arr_l[z];
        int x = arr[h];
        int i = (l - 1);
        int temp;
        for (int j = l; j <= h- 1; j++)
        {
            if (arr[j] <= x)
            {
                i++;
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
        temp = arr[i+1];
        arr[i+1] = arr[h];
        arr[h] = temp;
        int p = (i + 1);
        if (p-1 > l)
        {
            int ind = d_size++;
            arr_l[ind] = l;
            arr_h[ind] = p-1;
        }
        if ( p+1 < h )
        {
            int ind = d_size++;
            arr_l[ind] = p+1;
            arr_h[ind] = h;
        }
    }
}

void quickSortIterative (int arr[], int l, int h)
{
    int lstack[ h - l + 1 ], hstack[ h - l + 1];

    int top = -1, *d_d, *d_l, *d_h;

    lstack[ ++top ] = l;
    hstack[ top ] = h;

```

```

    cudaMalloc(&d_d, (h-l+1)*sizeof(int));
    cudaMemcpy(d_d, arr, (h-l+1)*sizeof(int), cudaMemcpyHostToDevice);

    cudaMalloc(&d_l, (h-l+1)*sizeof(int));
    cudaMemcpy(d_l, lstack, (h-l+1)*sizeof(int), cudaMemcpyHostToDevice);

    cudaMalloc(&d_h, (h-l+1)*sizeof(int));
    cudaMemcpy(d_h, hstack, (h-l+1)*sizeof(int), cudaMemcpyHostToDevice);

    int n_t = 1;
    int n_b = 1;
    int n_i = 1;
    while ( n_i > 0 )
    {
        partition<<<n_b,n_t>>>( d_d, d_l, d_h, n_i);
        int answer;
        cudaMemcpyFromSymbol(&answer, d_size, sizeof(int), 0,
cudaMemcpyDeviceToHost);
        if (answer < 1024)
        {
            n_t = answer;
        }
        else
        {
            n_t = 1024;
            n_b = answer/n_t + (answer%n_t==0?0:1);
        }
        n_i = answer;
        cudaMemcpy(arr, d_d, (h-l+1)*sizeof(int), cudaMemcpyDeviceToHost);
    }
}

```

```

int main()
{

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    int arr[10];
    for (int i = 0; i<10; i++)
    {
        arr[i] = rand ()%100;
    }
}

```

```

printf("Input Array : \n");
for(int i=0;i<10;++i)
    printf("%d ",arr[i]);

int n=sizeof(arr)/sizeof(*arr);

cudaEventRecord(start, 0);

quickSortIterative( arr, 0, n - 1 );

cudaEventRecord(stop,0);
cudaEventSynchronize(stop);

printf("\nSorted Array: \n");
for(int i=0;i<10;++i)
    printf("%d ",arr[i]);

float elapsedtime;
cudaEventElapsedTime(&elapsedtime, start, stop);
printf("\n CUDA Execution Time = %f ms", elapsedtime);

cudaEventDestroy(start);
cudaEventDestroy(stop);

return 0;
}

```

Output

```

[vk@Prithivi ~]$ nvcc ex6_time.cu
[vk@Prithivi ~]$ ./a.out

```

Input Array :
83 86 77 15 93 35 86 92 49 21

Sorted Array:
15 21 35 49 77 83 86 86 92 93

CUDA Execution Time = 0.396096 ms